Urban Mobility Data Explorer: Technical Report

NYC TLC yellow-taxi trips, January 2019 | Team 5: Orion Kabeza (lead), Emmanuel Rangira, Kmaster, Aubin Mfura, Djibril Mpamira | Flask + SQLite + vanilla JS (Chart.js, Leaflet)

1. Problem framing and dataset analysis

Dataset and context

The source is the NYC Taxi & Limousine Commission yellow-taxi record for January 2019: **7,667,792 trips** with 18 raw columns (pickup and dropoff timestamps, distance, rate code, pickup/dropoff zone IDs, payment type and itemised fares), joined to `taxi_zone_lookup.csv` (265 zones) and the `taxi_zones` polygons. The brief mentions .parquet and GeoJSON, but the files supplied are a 656 MB CSV and an ESRI shapefile, so the pipeline streams the CSV in 250,000-row chunks and reprojects the shapefile (NY State Plane feet, EPSG:2263) to WGS84 for Leaflet.

Data challenges

Seventeen cleaning rules run on every chunk; each excluded row is written to `logs/excluded_records.csv` with its original row number and every rule it broke. **203,344 rows (2.65%) were excluded, 7,464,448 kept.** A row can break several rules, so the counts below add up to more than the number of excluded rows.

Rule (reason code)	Rows	Rule (reason code)	Rows
passenger_count_invalid (not 1-6)	117,438	negative_charge	7,138
distance_non_positive	54,770	dropoff_not_after_pickup	6,294
duration_too_long (> 6 h)	20,534	implausible_speed (> 100 mph)	5,713
fare_non_positive	9,557	pickup_out_of_range (not Jan 2019)	537
total_non_positive	8,545	invalid_rate_code / fare_too_large / distance_too_large	252 / 34 / 32

Assumptions made while cleaning

- Thresholds: distance 100 mi or less, duration 6 h or less, speed 100 mph or less, fare $500 or less, 1-6 passengers, rate codes 1-6 (99 means unknown), payment types 1-6, pickup inside January 2019. All live in `pipeline/config.py`.
- `congestion_surcharge` is empty for most 2019 rows, so a blank means no surcharge (0). A missing store-and-forward flag becomes "N".
- Duplicates are found with a 64-bit hash of the identifying columns, kept across chunks in one array (8 bytes per row). None were found; the hash collision risk is negligible at this size.
- The meter records tips only for card payments (cash tips are stored as 0). `tip_pct` is therefore left NULL for non-card trips instead of averaging in false zeros.
- Shapefile quirks: zones 56 and 103 are split over several records (merged per LocationID); zones 57, 104, 105, 264 and 265 have no polygon (they stay in the tables, but have no map shape).

One unexpected observation

The most frequent route in the raw data is **"Unknown to Unknown"** (LocationIDs 264/265): **128,027 trips**, far ahead of the best real route (50,244). A ranking led by a placeholder says nothing about mobility, so the Top-K endpoints exclude zones 264 and 265, and the map leaves them unshaded. This shaped the API design.

2. System architecture and design decisions

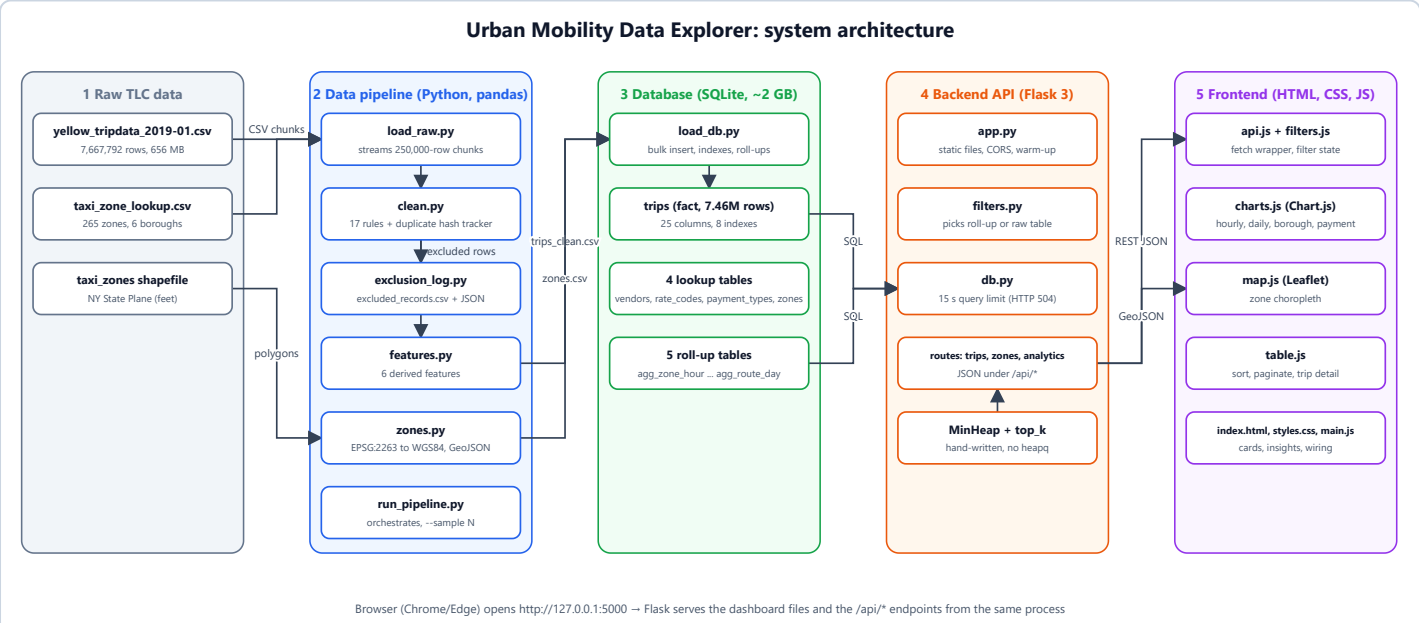


Figure 1. Data flows left to right: raw TLC files, the chunked pipeline, SQLite, the Flask API, then the browser. The editable draw.io source is `architecture.drawio`.

Stack choices

- **Flask:** a small REST layer that is easy to test (92 pytest tests) and serves the dashboard files too, so one command starts everything.
- **SQLite:** zero installation and a single ~2 GB file that holds all 7.46M trips, so the project runs on any machine from the README. The cost is a single writer, which is fine for a read-only dashboard.
- **Vanilla JS, Chart.js, Leaflet:** no build step; the filters, charts, choropleth map and sortable trip table all talk to `/api/*`.

Schema

A normalised star: the `trips` fact table (25 columns, `CHECK` constraints on passengers, distance, duration, hour) references the lookup tables `vendors`, `rate_codes`, `payment_types` and `zones` (twice: pickup and dropoff). Eight indexes cover what the filters use (pickup time, each zone, the route pair, distance, fare, hour, payment). Six **derived features** are computed once at load time: `trip_duration_min` (dropoff minus pickup), `avg_speed_mph` (a congestion proxy), `fare_per_mile` (compares short and long trips), `tip_pct` (card trips only), `pickup_hour` and `pickup_dow`. Five roll-up tables (`agg_zone_hour`, `agg_route_hour`,

agg_route_day and two distance/fare-banded versions) hold pre-summed counts and sums.

Trade-offs

- **Speed vs flexibility.** Grouping the raw table took 4-19 s per chart. Answering from the roll-ups takes **0.05-0.7 s**, but only for filter values on the band edges (distance 1, 2, 3, 5, 10, 20 mi; fares $5-$50). Any other value falls back to the raw table (about 3 s). A query stopped by the 15 s limit returns HTTP 504 with advice to narrow the filters.
- **Disk vs speed.** The roll-ups and stored features add redundancy on purpose; the database is bigger, the dashboard is interactive.
- **Full month vs sample.** We load all 7.46M cleaned rows (about 6 minutes) so statistics are honest; `--sample N` gives a quick trial.
- **Index discipline.** Mixed-direction `ORDER BY` and a unary + silently disabled indexes, so queries were rewritten and the planner hinted. A warm-up at start-up and per-card progressive rendering hide the remaining latency.
- **No deployment.** A 2 GB SQLite file does not suit free hosting; see future work.

3. Algorithmic logic and data structures

Problem. The dashboard needs the K busiest zones and the K most frequent routes out of 32,054 distinct routes, for any filter combination. Sorting every group costs O(n log n). We implemented a **binary min-heap by hand** (`backend/algorithms/min_heap.py`, list-based, parent = (i-1)/2, our own sift-up and sift-down) and a **bounded Top-K** on top of it (`topk.py`). No `heapq`, `sorted()`, `sort_values` or `Counter` is used. The heap never holds more than K items, and its root is always the weakest of the current top K, so a new item only has to beat the root to get in. The ordering key can be a tuple, for example `(trips, -zone_id)`, which breaks ties deterministically. The same routine ranks boroughs.

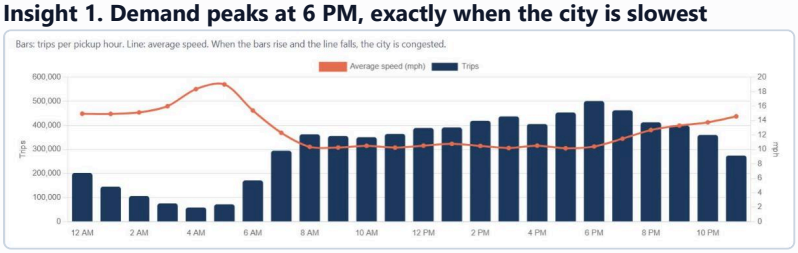
Complexity. `push`, `pop`, `replace_top`: O(log K). `peek`, `len`: O(1). `top_k` over n items: **time O(n log K)**, **space O(K)** beyond the input, against O(n log n) and O(n) for sorting. For K = 10 and n = 32,054, each item costs one comparison with the root, and only a few items cause a sift of at most four levels. Tests (`test_min_heap.py`) compare the result with a sorted oracle on random data and cover K of 0 or less, K larger than n, empty input and ties.

```
top_k(items, k, key):
    if k <= 0: return []
    heap <- empty min-heap ordered by key
    for item in items:                    # n iterations
        if size(heap) < k:
            push(heap, item)              # O(log k)
        else if key(item) > key(peek(heap)):   # O(1)
            replace_top(heap, item)       # evict weakest
    result <- array of size(heap)
    for pos from size(heap)-1 down to 0:
        result[pos] <- pop(heap)          # smallest first
    return result                         # strongest first

sift_up(i):
    while i > 0 and key(a[i]) < key(a[(i-1)/2]):
        swap(a[i], a[(i-1)/2]);  i <- (i-1)/2
sift_down(i):
    loop:
        s <- index of the smallest key among i, 2i+1, 2i+2
        if s = i: stop
        swap(a[i], a[s]);  i <- s
```

4. Insights and interpretation

Insight 1. Demand peaks at 6 PM, exactly when the city is slowest

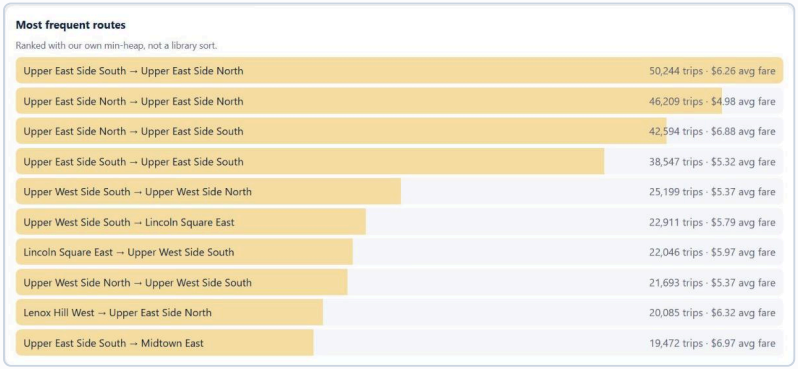


Dashboard card "demand versus traffic speed through the day": bars are trips per pickup hour, the line is average speed (mph).

How derived: `/api/analytics/hourly`, which reads the roll-up `agg_zone_hour` (`SUM(trips)` and `SUM(sum_speed)/SUM(speed_trips)` per pickup hour); weekday figures from `pickup_dow` on `trips`.

Interpretation. Trips peak at **6 PM with 500,885**, 8.5 times the 4 AM low (58,732). From 8 AM to 6 PM average speed sits at **10-11 mph**; at 4-5 AM it reaches 18-19 mph. Thursday is the busiest day (1.32M trips, 10.9 mph), Sunday the quietest (0.84M) and fastest (13.6 mph). Congestion tracks demand across a long daytime plateau, not one rush hour, so pricing or bus-lane policies should target roughly 8 AM to 7 PM.

Insight 2. Manhattan has the trips, Queens has the money



Dashboard card "Most frequent routes", ranked with our min-heap (Top-K, K = 10).

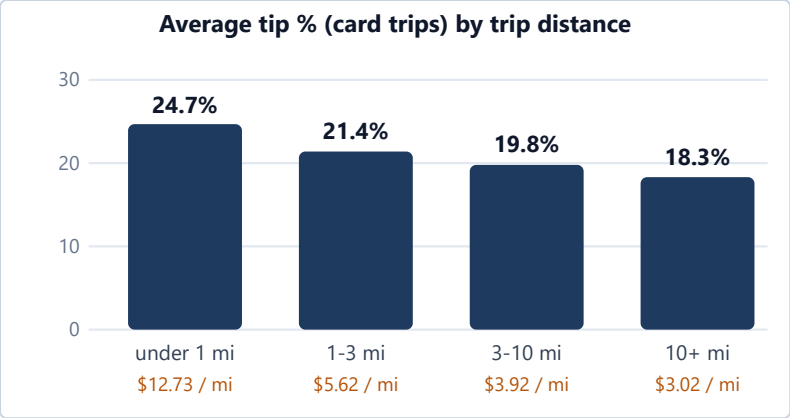
How derived: borough totals from `agg_zone_hour` joined to `zones`; routes from `/api/analytics/top-routes`, ranked by the hand-written heap over 32,054 routes.

Interpretation. Manhattan starts **90.8%** of trips but only 78.8% of metered fares (average fare $10.56 over 2.2 mi). Queens starts 5.9% of trips yet **17.3%** of fares (average $35.57 over 11.6 mi), driven by airport runs: JFK flat-fare trips average $52, and the two darkest zones outside Manhattan on the map are the airports. The ten busiest real routes are all short Upper East and Upper West Side hops costing $5-$7, 309,000 trips in total. Volume comes from short hops; revenue per trip comes from long airport runs.



Dashboard map: pickups per zone, 0 (pale) to 326,051 (dark).

Insight 3. Short trips tip more, and cash tips are invisible



Bars: average `tip_pct`. Orange: average `fare_per_mile` in the band. Based on 5.36M card trips.

How derived: SQL on `trips`: card trips (payment type 1) grouped by distance band, averaging the derived columns `tip_pct` and `fare_per_mile`; the payment mix comes from `/api/analytics/payment-types`.

Interpretation. 71.9% of trips are paid by card and tip **21.8%** on average; the 27.8% paid in cash record no tip at all, so true tipping is under-reported. The tip percentage falls from 24.7% on trips under a mile to 18.3% beyond ten miles, while fare per mile collapses from $12.73 to $3.02. Riders appear to tip a roughly flat amount, so short hops are worth far more per mile to drivers, which also explains why they dominate the busiest-route ranking.

5. Reflection and future work

Technical challenges

- **Scale.** At 7.46M rows, a plain GROUP BY made every chart wait 4-19 s. Roll-up tables, query hints, a 15 s query limit, a warm-up and progressive card rendering brought the dashboard to interactive speed.
- **Messy spatial data.** Reprojecting State Plane feet, merging split zones and handling five zones without polygons took more care than the cleaning rules themselves.
- **Resources.** The full build needs about 5 GB of free disk and several minutes; the README documents a `--sample` mode for quick checks.
- **Known weakness.** Chart.js is loaded from a public CDN; on a network that blocks it the charts stay blank. Vendoring the file inside `frontend/` removes that risk.

Team challenges

- Five people built five modules against a shared API contract. Small slips in hand-typed code (a misspelled column, a missing `__name__`, indentation) only appeared at integration, when the pieces were merged.
- We answered with compile and lint checks, a 92-test suite (every cleaning rule, the heap, the API and the filters) and fix pull requests. Lesson: run the whole pipeline and the tests before every merge, not only per module.

Next steps for a real product

- **Deploy:** PostgreSQL (or a managed database) behind gunicorn and nginx in a container, with the roll-ups refreshed by the pipeline.
- **More data:** load the other 11 months and later years for seasonality and year-on-year trends; partition by month.
- **Richer context:** join weather, events and holidays; add pickup-to-dropoff flow maps and speed heat maps per zone.
- **Anomaly detection:** flag suspicious fares and speeds per zone and hour with a robust statistic instead of fixed thresholds; keep the exclusion log for audit.
- **Quality:** continuous integration running `compileall` and `pytest` on every pull request, a locally served Chart.js, keyboard and screen-reader testing of the dashboard, and a better treatment of cash tips (an estimate flagged as such).